Doc. no. P0174R2
Date: 2016-06-23
Project: Programming Language C++
Audience: Library Evolution Working Group
Reply to: Alisdair Meredith <ameredith1@bloomberg.net>

Deprecating Vestigial Library Parts in C++17

Table of Contents

Revision History

Revision 0

Original version of the paper for the 2016 pre-Jacksonville mailing.

Revision 1

Following Jacksonville review.

- Do not deprecate algorithms with incomplete ranges
- Do not deprecate `value_compare` predicates
- Deprecate `is_literal_type` now
- Deprecate even more of `std::allocator`
- Deprecate temporary buffer APIs now
- Deprecate `raw_storage_iterator` now

Revision 2

Following Jacksonville review.

- Clean up editorial nits from LWG review
- Better wording to introduce deprecated class specializations

Introduction

A number of features of the C++ Standard library have been surpassed by additions over the years, or we have learned do not serve their intended purpose as well as originally expected. This paper proposed deprecating features where better, simpler, or clearer options are available.

Recommendations for Deprecation

There is likely to be a reasonable amount of user code using all of the feature below in production today, so it would be premature to remove them from the standard. However, with best best practice advancing and superior or clearer options available within the standard itself, now would be a good time to deprecate these features, directing users to the preferred alternative instead.

Deprecate `std::iterator`

As an aid to writing iterator classes, the original standard library supplied the `iterator` class template to automate the declaration of the five typedefs expected of every iterator by `iterator_traits`. This was then used in the library itself, for instance in the specification of `std::ostream_iterator`:

```
template <class T, class charT = char, class traits = char_traits<charT> >
class ostream_iterator:
  public iterator<output_iterator_tag, void, void, void, void>;
```

The long sequence of `void` arguments is much less clear to the reader than simply providing the expected typedefs in the class definition itself, which is the approach taken by the current working draft, following the pattern set in C++14 where we deprecated the derivation throughout the library of functors from `unary_function` and `binary_function`.

In addition to the reduced clarity, the `iterator` template also lays a trap for the unwary, as in typical usage it will be a dependent base class, which means it will not be looking into during name lookup from within the class or its member functions. This leads to surprised users trying to understand why the following simple usage does not work:

```
#include <iterator>

template <typename T>
struct MyIterator : std::iterator<std::random_access_iterator_tag, T> {
   value_type data;  // Error: value_type is not found by name lookup

   // ... implementations details elided ...
};
```

The reason of clarity alone was sufficient to persuade the LWG to update the standard library specification to no longer mandate the standard iterator adapators as deriving from `std::iterator`, so there is no further use of this template within the standard itself. Therefore, it looks like a strong candidate for deprecation.

Update from Jacksonville, 2016:

Poll: Deprecate `iterator` for C++17??

SF	F	N	A	SA
6	10	1	0	0

Deprecate the redundant members of `std::allocator`

Many members of `std::allocator` redundantly duplicate behavior that is otherwise produced by `std::allocator_traits<allocator<T>>`, and could safely be removed to simplify this class. Further, `addressof` as a free function supersedes `std::allocator<T>::address` which requires an `allocator` object of the right type. Finally, the `reference` type aliases were initially provided as an expected means for extension with other allocators, but turned out to not serve a useful purpose when we specified the allocator requirements (17.6.3.5 [allocator.requirements]).

While we cannot remove these members without breaking backwards compatibility with code that explicitly used this allocator type, we should not be recommending their continued use. If a type wants to support generic allocators, it should access the allocator's functionality through `allocator_traits` rather than directly accessing the allocator's members - otherwise it will not properly support allocators that rely on the traits to synthesize the default behaviors. Similarly, if a user does not intend to support generic allocators, then it is much simpler to directly invoke `new`, `delete`, and assume the other properties of `std::allocator` such as pointer-types directly.

While the `is_always_equal` trait might be implicitly synthesized from the trait `is_empty_v<allocator<T>>`, there appears to be no specified guarantee that `std::allocator` specializations be empty classes (although this is widely expected) so the trait remains explicitly specialized to provide the necessary guarantee, even for implementations that choose to add some data to `std::allocator` for debug, profiling, or other purposes.

Similarly, `std::allocator<void>` is defined so that various template rebinding tricks could work in the original C++98 library, but it is not an actual allocator, as it lacks both `allocate` and `deallocate` member functions, which cannot be synthesized by default from `allocator_traits`. That need went away with C++11 and the `void_pointer` and `const_void_pointer` type aliases in `allocator_traits`. However, we continue to specify it in order to avoid breaking old code that has not yet been upgraded to support generic allocators, per C++11.

This paper recommends deprecating `std::allocator<void>` and the redundant members of the `std::allocator` class template, and moving their declarations and definitions to Annex D, just as the old iostreams members were handled in the original 1998 standard.

One potential concern is whether removing these members in the future might cause a compile-time performance regression, if the library is expected to deduce the default behavior in each case. The proposed Zombie Names clause from P0090R0 and incorporated in P0005R3 would allow vendors to retain these names indefinitely, if that were a concern. However, the optimal (compile-time) solution for the library implementation is to provide a partial specialization for `std::allocator_traits<std::allocator<T>>`, and so entirely avoid the cost of evaluating complex template machinery for the default allocator in the standard library.

It was also observed, while writing this paper, that the allocator propagation traits for `std::allocator<void>` do not match those for `std::allocator<T>`. It was decided not to provide a drive-by fix, as `std::allocator<void>` is not a true allocator, and so should not end up in code relying on those traits. If the desision is made not to deprecate this specialization though, it could be worth revisitting the issue, just to address consistency and a lack of surprise.

Update from Jacksonville, 2016:

Poll: Deprecate Redundant `allocator` Parts for C++17??

SF	F	N	A	SA
9	7	1	0	0

Poll: Deprecate move aggressively (`max_size` and `allocate` hint) for C++17??

SF	F	N	A	SA
5	7	4	1	0

Deprecate the `is_literal` Trait

The `is_literal` type trait offers negligible value to generic code, as what is really needed is the ability to know that a specific construction would produce constant initialization. The core term of a *literal type* having at least one `constexpr` constructor is too weak to be used meaningfully.

However, the (already implemented) trait does no harm, and allows compile-time introspection for which core-language type-categories a given template parameter might satisfy. Until the Core Working Group retire the notion of a literal type, the corresponding library trait should be preserved.

The next step towards removal (after deprecation) would be to write a paper proposing to remove the term from the core language while deprecating/removing the type trait.

Update from Jacksonville, 2016:

Poll: Deprecate `is_literal` for C++17?

SF	F	N	A	SA
9	4	3	1	0

Additional Candidates

Several additional aspects of the library were looked at, but decided against a recommendation for deprecation at this point. They are listed here, with rationale, in case others are more motivated. With appropriate additions to the standard, these may become stronger deprecation candidates for the next standard.

If members of the Library Working Groups would like to see any of these additional libraries deprecated, the author is happy to extend the proposal, but feels that they go beyond the remit of this paper deprecating only features where a preferable alternative is already available in the standard.

Reconsider `vector<bool>` Partial Specialization

There has been a long history of the `bool` partial specialization of `std::vector` not satisfying the container requirements, and in particular, its iterators not satisfying the requirements of a random access iterator. A previous attempt to deprecate this container was rejected for C++11, N2204.

One of the reasons for rejection is that it is not clear what it would mean to deprecate a particular specialization of a template. That could be addressed with careful wording. The larger issue is that the (packed) specialization of `vector<bool>` offers an important optimization that clients of the standard library genuinely seek, but would not longer be available. It is unlikely that we would be able to deprecate this part of the standard until a replacement facility is proposed and accepted, such as N2050. Unfortunately, there are no such revised proposals currently being offered to the Library Evolution Working Group.

Reconsider the Temporary Buffer APIs

Library clause 20.7.11 **[temporary.buffer]** provides an API intended to efficiently create a *small* ammount of additional storage for local, short-term use. The inspiration behind the API is to use some magic to create a tiny buffer on the stack.

This API would be considered an incomplete thought were it proposed today. As a functional API it lacks exception safety if the function allocating the buffer leaks, yet we offer no RAII-like wrappers to promote safe use.

It has been suggested that all current implementation of this API actually do not perform a more efficient allocation than the regular `new` operator, and, if that is genuinely the case, we should seriously consider deprecating this facility. Otherwise, we should probably complete the design with an appropriate guard/wrapper class, and encourage vendors to deliver on missed optimization opportunities.

It is possible the need for this API could be overtaken by delivering on a genuine C++ API or data structure for allocating off the stack, such as was proposed by `std::dynarray` in N3662. However, the Evolution Working Group has not been keen on further work in this direction, and the Arrays TS appears to have stalled, if not abandoned entirely.

Update from Jacksonville, 2016:

Poll: Deprecate the temporary buffer API?

SF	F	N	A	SA
5	10	2	1	0

Reconsider raw storage iterators

Like the temporary buffer API, the `raw_storage_iterator` (20.7.10 **[storage.iterator]**) and uninitialized algorithms (20.7.12 **[specialized.algorithms]**) are incomplete thoughts, although they appeared complete at the time of adoption.

One obvious shortcoming of the iterator adapter is that it could easily query the type of element the adapated iterator is prepared to construct via `iterator_traits` and provide a default template argument for the second parameter, simplifying use. That no such proposal has come forward in almost 20 years suggests that this is not a widely used component. Similarly, a type-deducing factory function would be a natural extension, especially with the advent of `auto` in C++11, yet no such proposal has materialized.

A deeper concern arises from the completion of allocator support in C++11. There is now a common requirement that construting an object in a region of allocated memory should invoke the `construct` method of the corresponding allocator. This is not possible with the current iterator, and undermines its use as an implementation detail in users writing generic containers. However, no-one seems motivated to invest the time in proposing an allocator-aware raw storage iterator, nor adding allocator-awareness to the uninitialized memory algorithms.

Without a clear alternative to replace them, there is no clear deprecation path other than simply looking to drop support for the whole idea in some future standard. That goes beyond the intent of this paper.

Update from Jacksonville, 2016:

Poll: Deprecate raw storage iterators (at next opportunity)?

SF	F	N	A	SA
2	12	2	2	0

Reconsider algorithms taking half an input range

The standard library has several algorithms that read from two ranges in order to determine their result. In most cases, the original C++98 standard fully specified the first range with a pair of iterators, and supplied only the first iterator for the second range, with a narrow contract requirement that the second range be at least as large as the first. For the C++14 standard, these algorithms were overloaded with a second form where the second range is also fully specified by a pair of iterators, partly so that the length of the second sequence can be taken into account (when it is shorter), and partly because this form is less liable to misuse leading to buffer overruns and other security risks. See **N3671** **Making non-modifying sequence operations more robust** for more details.

Given the security risks associated with the original algorithms, this paper recommends deprecating the form of any algorithm where a second input range is specified by only one iterator. Note that algorithms using a single iterator for an output range continue to be supported, as there is no (standard) way to specify the end of an output iteration sequence, such as defined by an `std::ostream_iterator`.

Update from Jacksonville, 2016:

Poll: Deprecate the listed algorithms with half-specified ranges?

SF	F	N	A	SA
3	6	3	5	1

There is no consensus to deprecate at this point, although interest remains to revisit for a future standard, probably after the Ranges TS has landed.

Reconsider `value_compare` predicates

The standard containers `map` and `multimap` both contain an identical class template `value_compare` that is a functor which can be used to determine the relative order of two elements in that container. These are provided as elements in a map are ordered by only the key value, rather than the whole value, so the predicate supplied to the container compares the key type, not the actual element type.

In practice these functors are not used by the containers, as they are often required to compare keys with values for elements that have not yet been inserted into the map, and may not have been created as a `pair` yet. Meanwhile, the cost of having a nested class type, rather than an alias to a functor with the same behavior, means that every associative container with a different allocator has an identical copy of this class, but with a different name mangling so that the duplicate functionality cannot be elided or merged. The class name itself will have a longer mangling than necessary, which would add up if these functors were genuinely useful and saw much use in practice.

However, it is not clear that these functors do see much actual use. For example, a quick search with Google Code search while preparing this paper turned up 29 hits, which were all standard library implementations, or test drivers for standard library implementations, and a similar result for the `value_comp()` function that returns an appropriately constructed functor from the container for the client to use.

This paper recommends deprecating the `value_compare` member classes of the associative containers, and moving their declarations and definitions to Annex D, just as the old iostreams members were handled in the original 1998 standard. It further recommends making it unspecified whether these members are provided as member-classes, per the text of the standard, or as aliases to a class with the same interface, allowing, but not requiring, vendors to choose to provide these members in a less redundant (but ABI-breaking) manner.

Update from Jacksonville, 2016:

Poll: Replace value_compare by unspecified type with the same semantics (for after C++17)?

SF	F	N	A	SA
5	10	2	0	0

Proposed Wording

Amend existing library clauses as below:

20.7.2 Header <memory> synopsis [memory.syn]

```
namespace std {
  // ...

  // 20.7.9, the default allocator:
  template <class T> class allocator;
  template <> class allocator<void>;
  template <class T, class U>
    bool operator==(const allocator<T>&, const allocator<U>&) noexcept;
  template <class T, class U>
    bool operator!=(const allocator<T>&, const allocator<U>&) noexcept;

  // 20.9.10, raw storage iterator:
  template <class OutputIterator, class T> class raw_storage_iterator;

  // 20.9.11, temporary buffers:
  template <class T>
    pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
  template <class T>
    void return_temporary_buffer(T* p);

  // ...
}
```

20.7.9 The default allocator [default.allocator]

1. All specializations of the default allocator satisfy the allocator completeness requirements 17.6.3.5.1.

```
namespace std {
  template <class T> class allocator;

  // specialize for void:
  template <> class allocator<void> {
  public:
    typedef void* pointer;
    typedef const void* const_pointer;
    // reference-to-void members are impossible.
    typedef void value_type;
    template <class U> struct rebind { typedef allocator<U> other; };
  };

  template <class T> class allocator {
   public:
    typedef size_t     size_type;
    typedef ptrdiff_t difference_type;
    typedef T*         pointer;
    typedef const T*   const_pointer;
    typedef T&         reference;
    typedef const T&   const_reference;
    typedef T          value_type;
    template <class U> struct rebind { typedef allocator<U> other; };
    typedef true_type propagate_on_container_move_assignment;
    typedef true_type is_always_equal;
    allocator() noexcept;
    allocator(const allocator&) noexcept;
    template <class U> allocator(const allocator<U>&) noexcept;
    ~allocator();
    pointer address(reference x) const noexcept;
    const_pointer address(const_reference x) const noexcept;
    pointerT* allocate(
      size_type, allocator<void>::const_pointer hint = 0);
    void deallocate(pointerT* p, size_type n);
    size_type max_size() const noexcept;
    template<class U, class... Args>
      void construct(U* p, Args&&... args);
    template <class U>
      void destroy(U* p);
  };
}
```

20.7.9.1 allocator members [allocator.members]

1. Except for the destructor, member functions of the default allocator shall not introduce data races (1.10) as a result of concurrent calls to those member functions from different threads. Calls to these functions that allocate or deallocate a particular unit of storage shall occur in a single total order, and each such deallocation call shall happen before the next allocation (if any) in this order.

   ~~`pointer address(reference x) const noexcept;`~~

2. ~~*Returns:* The actual address of the object referenced by `x`, even in the presence of an overloaded `operator&`.~~

   ~~`const_pointer address(const_reference x) const noexcept;`~~

3. ~~*Returns:* The actual address of the object referenced by `x`, even in the presence of an overloaded `operator&`.~~

   ~~`pointer`~~`T*` `allocate(size_type n,` ~~`allocator::const_pointer hint = 0`~~`);`

4. ~~[*Note:* In a container member function, the address of an adjacent element is often a good choice to pass for the hint argument. *— end note*]~~
5. *Returns:* A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`. It is implementation-defined whether over-aligned types are supported (3.11).
6. *Remark:* the storage is obtained by calling `::operator new(std::size_t)` (18.6.1), but it is unspecified when or how often this function is called. ~~The use of `hint` is unspecified, but intended as an aid to locality if an implementation so desires.~~
7. *Throws:* `bad_alloc` if the storage cannot be obtained.

   `void deallocate(`~~`pointer`~~`T*` `p, size_type n);`

8. *Requires:* `p` shall be a pointer value obtained from `allocate()`. `n` shall equal the value passed as the first argument to the invocation of allocate which returned `p`.
9. *Effects:* Deallocates the storage referenced by `p`.
10. *Remarks:* Uses `::operator delete(void*, std::size_t)` (18.6.1), but it is unspecified when this function is called.

    ~~`size_type max_size() const noexcept;`~~

11. ~~*Returns:* The largest value `N` for which the call `allocate(N,0)` might succeed.~~

    ~~`template <class U, class... Args>`~~
    ~~`void construct(U* p, Args&&... args);`~~

12. ~~*Effects:* `::new((void *)p) U(std::forward<Args>(args)...)`~~

    ~~`template <class U>`~~
    ~~`void destroy(U* p);`~~

13. ~~*Effects:* `p->~U()`~~

20.9.10 Raw storage iterator [storage.iterator]

1. ~~`raw_storage_iterator` is provided to enable algorithms to store their results into uninitialized memory. The template parameter `OutputIterator` is required to have its `operator*` return an object for which `operator&` is defined and returns a pointer to `T`, and is also required to satisfy the requirements of an output iterator (24.2.4).~~

   ~~~~
   namespace std {
     template <class OutputIterator, class T>
     class raw_storage_iterator {
     public:
       typedef output_iterator_tag iterator_category;
       typedef void value_type;
       typedef void difference_type;
       typedef void pointer;
       typedef void reference;

       explicit raw_storage_iterator(OutputIterator x);

       raw_storage_iterator& operator*();
       raw_storage_iterator& operator=(const T& element);
       raw_storage_iterator& operator=(T&& element);
       raw_storage_iterator& operator++();
       raw_storage_iterator  operator++(int);
       OutputIterator base() const;
     };
   }
   ~~~~

   ~~`explicit raw_storage_iterator(OutputIterator x);`~~

2. ~~*Effects:* Initializes the iterator to point to the same value to which `x` points.~~

~~raw_storage_iterator& operator*();~~

3. ~~*Returns:* *this~~

~~raw_storage_iterator& operator=(const T& element);~~

4. ~~*Requires:* T shall be CopyConstructible.~~
5. ~~*Effects:* Constructs a value from element at the location to which the iterator points.~~
6. ~~*Returns:* A reference to the iterator.~~

~~raw_storage_iterator& operator=(T&& element);~~

7. ~~*Requires:* T shall be MoveConstructible.~~
8. ~~*Effects:* Constructs a value from std::move(element) at the location to which the iterator points.~~
9. ~~*Returns:* A reference to the iterator.~~

~~raw_storage_iterator& operator++();~~

10. ~~*Effects:* Pre-increment: advances the iterator and returns a reference to the updated iterator.~~

~~raw_storage_iterator operator++(int);~~

11. ~~*Effects:* Post-increment: advances the iterator and returns the old value of the iterator.~~

~~OutputIterator base() const;~~

12. ~~*Returns:* An iterator of type OutputIterator that points to the same value as *this points to.~~

~~20.9.11 Temporary buffers [temporary.buffer]~~

~~template <class T>~~
~~pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;~~

1. ~~*Effects:* Obtains a pointer to uninitialized, contiguous storage for *N* adjacent objects of type T, for some non-negative number *N*. It is implementation-defined whether over-aligned types are supported (3.11).~~
2. ~~*Remarks:* Calling get_temporary_buffer with a positive number n is a non-binding request to return storage for n objects of type T. In this case, an implementation is permitted to return instead storage for a non-negative number *N* of such objects, where *N* != n (including *N* == 0). [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations of its memory management. - *end note*]~~
3. ~~*Returns:* If n <= 0 or if no storage could be obtained, returns a pair P such that P.first is a null pointer value and P.second == 0; otherwise returns a pair P such that P.first refers to the address of the uninitialized storage and P.second refers to its capacity *N* (in the units of sizeof(T)).~~

~~template <class T> void return_temporary_buffer(T* p);~~

1. ~~*Effects:* Deallocates the storage referenced by p.~~
2. ~~*Requires:* p shall be a pointer value returned by an earlier call to get_temporary_buffer that has not been invalidated by an intervening call to return_temporary_buffer(T*).~~
3. ~~*Throws:* Nothing.~~

20.10.2 Header `<type_traits>` synopsis [meta.type.synop]

```
namespace std {
  // Start of header elided...

  // 20.10.4.3, type properties:
  template <class T> struct is_const;
  template <class T> struct is_volatile;
  template <class T> struct is_trivial;
  template <class T> struct is_trivially_copyable;
  template <class T> struct is_standard_layout;
  template <class T> struct is_pod;
  ~~template <class T> struct is_literal_type;~~
  template <class T> struct is_empty;
  template <class T> struct is_polymorphic;
  template <class T> struct is_abstract;
  template <class T> struct is_final;

  // Rest of header elided...
}
```

20.10.4.3 Type properties [meta.unary.prop]

Table 49 - Type property predicates

Template	Condition	Preconditions
~~`template <class T> struct is_literal_type;`~~	~~`T` is a literal type (3.9)~~	~~`remove_all_extents_t<T>` shall be a complete type or (possibly cv-qualified) `void`.~~

24.3 Header `<iterator>` synopsis [iterator.synopsis]

```
namespace std {
  // 24.4, primitives:
  template<class Iterator> struct iterator_traits;
  template<class T> struct iterator_traits<T*>;

  template<class Category, class T, class Distance = ptrdiff_t,
           class Pointer = T*, class Reference = T&> struct iterator;

  // Rest of header elided...
}
```

24.4.2 Basic iterator [iterator.basic]

1. The `iterator` template may be used as a base class to ease the definition of required types for new iterators.

```
namespace std {
  template<class Category, class T, class Distance = ptrdiff_t,
    class Pointer = T*, class Reference = T&>
  struct iterator {
    typedef T         value_type;
    typedef Distance  difference_type;
    typedef Pointer   pointer;
    typedef Reference reference;
    typedef Category  iterator_category;
  };
}
```

Insert new clauses into Annex D:

D.v The default allocator [depr.default.allocator]

1. The following members and explicit class template specialization are defined in addition to those specified in Clause 20:

```
namespace std {
  template <> class allocator<void> {
  public:
    typedef void* pointer;
    typedef const void* const_pointer;
    // reference-to-void members are impossible.
    typedef void value_type;
    template <class U> struct rebind { typedef allocator<U> other; };
  };

  template <class T> class allocator {
  public:
    typedef size_t    size_type;
    typedef ptrdiff_t difference_type;
    typedef T*        pointer;
    typedef const T*  const_pointer;
    typedef T&        reference;
    typedef const T&  const_reference;
    template <class U> struct rebind { typedef allocator<U> other; };

    T* address(reference x) const noexcept;
    const T* address(const_reference x) const noexcept;

    T* allocate(size_t, const void* hint);

    template<class U, class... Args>
      void construct(U* p, Args&&... args);
    template <class U>
      void destroy(U* p);

    size_t max_size() const noexcept;
  };
}
```

`T* address(reference x) const noexcept;`

2. *Returns:* The actual address of the object referenced by `x`, even in the presence of an overloaded `operator&`.

```
const T* address(const_reference x) const noexcept;
```

3. *Returns:* The actual address of the object referenced by `x`, even in the presence of an overloaded `operator&`.

```
T* allocate(size_t, const void* hint);
```

4. *Returns:* A pointer to the initial element of an array of storage of size `n * sizeof(T)`, aligned appropriately for objects of type `T`. It is implementation-defined whether over-aligned types are supported (3.11).
5. *Remark:* the storage is obtained by calling `::operator new(std::size_t)` (18.6.1), but it is unspecified when or how often this function is called.
6. *Throws:* `bad_alloc` if the storage cannot be obtained.

```
template <class U, class... Args>
  void construct(U* p, Args&&... args);
```

7. *Effects:* `::new((void *)p) U(std::forward<Args>(args)...)`

```
template <class U>
  void destroy(U* p);
```

8. *Effects:* `p->~U()`

```
size_type max_size() const noexcept;
```

9. *Returns:* The largest value `N` for which the call `allocate(N)` might succeed.

D.w Raw storage iterator [depr.storage.iterator]

The header `<memory>` has the following addition:

```
namespace std {
  template <class OutputIterator, class T>
  class raw_storage_iterator {
  public:
    typedef output_iterator_tag iterator_category;
    typedef void value_type;
    typedef void difference_type;
    typedef void pointer;
    typedef void reference;

    explicit raw_storage_iterator(OutputIterator x);

    raw_storage_iterator& operator*();
    raw_storage_iterator& operator=(const T& element);
    raw_storage_iterator& operator=(T&& element);
    raw_storage_iterator& operator++();
    raw_storage_iterator  operator++(int);
    OutputIterator base() const;
  };
}
```

1. `raw_storage_iterator` is provided to enable algorithms to store their results into uninitialized memory. The template parameter `OutputIterator` is required to have its `operator*` return an object for which `operator&` is defined and returns a pointer to `T`, and is also required to satisfy the requirements of an output iterator (24.2.4).

```
explicit raw_storage_iterator(OutputIterator x);
```

2. *Effects:* Initializes the iterator to point to the same value to which `x` points.

```
raw_storage_iterator& operator*();
```

3. *Returns:* `*this`

```
raw_storage_iterator& operator=(const T& element);
```

4. *Requires:* `T` shall be `CopyConstructible`.
5. *Effects:* Constructs a value from `element` at the location to which the iterator points.
6. *Returns:* A reference to the iterator.

```
raw_storage_iterator& operator=(T&& element);
```

7. *Requires:* `T` shall be `MoveConstructible`.
8. *Effects:* Constructs a value from `std::move(element)` at the location to which the iterator points.
9. *Returns:* A reference to the iterator.

```
raw_storage_iterator& operator++();
```

10. *Effects:* Pre-increment: advances the iterator and returns a reference to the updated iterator.

```
raw_storage_iterator operator++(int);
```

11. *Effects:* Post-increment: advances the iterator and returns the old value of the iterator.

```
OutputIterator base() const;
```

12. *Returns:* An iterator of type `OutputIterator` that points to the same value as `*this` points to.

D.x Temporary buffers [depr.temporary.buffer]

1. The header `<memory>` has the following additional functions:

```
namespace std {
  template <class T>
    pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
  template <class T>
    void return_temporary_buffer(T* p);
}
```

```
template <class T>
  pair<T*, ptrdiff_t> get_temporary_buffer(ptrdiff_t n) noexcept;
```

2. *Effects:* Obtains a pointer to uninitialized, contiguous storage for *N* adjacent objects of type `T`, for some non-negative number *N*. It is implementation-defined whether over-aligned types are supported (3.11).
3. *Remarks:* Calling `get_temporary_buffer` with a positive number `n` is a non-binding request to return storage for `n` objects of type `T`. In this case, an implementation is permitted to return instead storage for a non-negative number *N* of such objects, where *N* `!= n` (including *N* `== 0`). [*Note:* The request is non-binding to allow latitude for implementation-specific optimizations of its memory management. - *end note*]
4. *Returns:* If `n <= 0` or if no storage could be obtained, returns a pair `P` such that `P.first` is a null pointer value and `P.second == 0`; otherwise returns a pair `P` such that `P.first` refers to the address of the uninitialized storage and `P.second` refers to its capacity *N* (in the units of `sizeof(T)`).

```
template <class T> void return_temporary_buffer(T* p);
```

1. *Effects:* Deallocates the storage referenced by `p`.
2. *Requires:* `p` shall be a pointer value returned by an earlier call to `get_temporary_buffer` that has not been invalidated by an intervening call to `return_temporary_buffer(T*)`.
3. *Throws:* Nothing.

D.y Deprecated Type Traits [depr.meta.type]

1. The header `<type_traits>` has the following addition:

```
namespace std {
  template <class T> struct is_literal_type;
}
```

2. *Requires:* `remove_all_extents_t<T>` shall be a complete type or (possibly cv-qualified) `void`.
3. *Effects:* `is_literal_type` has a base-characteristic of `true_type` if `T` is a literal type (3.9), and a base-characteristic of `false_type` otherwise.

D.z Basic iterator [depr.iterator.basic]

1. The header `<iterator>` has the following addition:

```
namespace std {
  template<class Category, class T, class Distance = ptrdiff_t,
    class Pointer = T*, class Reference = T&>
  struct iterator {
    typedef T         value_type;
    typedef Distance  difference_type;
    typedef Pointer   pointer;
    typedef Reference reference;
    typedef Category  iterator_category;
  };
}
```

2. The `iterator` template may be used as a base class to provide the type alias members required for the definition of an iterator type.
3. [*Note:* If the new iterator type is a class template, then these aliases will not be visible from within the iterator

class's template definition, but only to callers of that class - *end note*]

References

- N3671 Making non-modifying sequence operations more robust: Revision 2, Mike Spertus, Attila Pall
- P0090R0 Removing result_type, etc., Stephan T. Lavavej